

Mapeamento em escala

Utilizar o **QGIS** para gerar centenas de mapas

Repositório Github:

`github.com/paulbradshaw/QGIS_param`

Em 1 hora saberá:

- Como utilizar o **Python do QGIS** funcionalidades
- Como utilizar a **parametrização** para gerar um mapa para cada área/parceiro/cidade
- Dicas sobre como usar **IA** para ajudar na programação

Cenário:

A BBC Shared Data Unit está a trabalhar numa reportagem sobre defesas contra cheias. Será enviada para redações em todo o Reino Unido. Queremos gerar uma imagem de mapa para a redação de cada cidade, mostrando o estado das defesas contra cheias na sua área.

**Como crio um
mapa?**

Faça agora:

**Encontre o ficheiro
shapefile**

Importar para o QGIS

Siga o guia no repositório

1. Vá a [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) e descarregue o ficheiro chamado `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Descomprima esse ficheiro
3. Abra o QGIS e crie um novo projeto (*Project > New*). Guarde-o na mesma pasta.
4. Selecione *Layer > Add Layer > Add vector layer*.
5. Na janela que aparece, clique em ... junto à secção Fonte (onde diz *Vector Dataset(s)*) e localize o ficheiro na pasta descarregada que termina em **.shp**. Clique em **Add**. Deverá aparecer em fundo atrás da janela. Clique em **OK** e depois em **Close** para regressar à janela principal.
6. In the *Browser* na parte superior esquerda do ecrã, desloque-se para baixo até à secção XYZ Tiles. Clique duas vezes em OpenStreetMap para adicionar essa camada de base ao seu projeto (clique em OK para aceitar a mudança para o mesmo CRS). Esta deverá aparecer agora na sua área de Camadas.
7. Clique e arraste a camada OSM para a colocar abaixo da camada de defesas contra cheias.

Personalizar a aparência

1. Clique com o botão direito na camada de defesas contra cheias e selecione **Properties...**
2. Selecione **Symbology** para alterar a cor, opacidade, largura, estilo etc. das linhas/formas

**Como exporto
uma imagem?**

Faça agora:

Faça zoom numa área

Project > New Print

Layout...

Criar um layout (vamos automatizar isto)

1. Faça zoom na parte do mapa da qual quer criar uma imagem (ex.: com zoom numa cidade)
2. Vá a **Project > New Print Layout...** Dê-lhe um nome (ex.: cityzoom) e clique em **OK**
3. A janela de layout aparecerá em branco. Clique no botão Adicionar Mapa à esquerda (ícone de página com um mais verde)
4. Clique e arraste para desenhar a área onde quer que o mapa apareça.
5. Vá ao separador **Item Properties** no lado direito e desloque-se para baixo até **Position and Size**.
Altere para as dimensões que pretende para a imagem (mude a medida para pixels se for para web).
6. Mude para o separador **Layout** ao lado desse, e desloque-se para baixo até ao botão **Resize** do layout. Clique nele. A imagem deverá agora ocupar toda a tela (pode fazer zoom usando os botões de zoom no canto superior esquerdo)
7. Quando estiver satisfeito com a imagem, vá a **Layout > Export as Image**
8. Feche a janela de Composição de Impressão e regresse ao projeto QGIS



Como faço isso
várias vezes?

Se tem de fazer algo mais do que uma vez...

Plugins > Python Console

Escreva/abra Python na Consola **abaixo do mapa.**

*Clique no botão **Show Editor** para abrir/editar ficheiros .py*

PyQGIS library ([documentação](#))

Faça agora:

Descarregue the .py file

Abra-o na sua Consola

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>



Untitled-0 X

1

Coordinate

357074,195749



Scale

1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(Passos na parte 3 do guia no repositório)

1. Vá ao [raw file](#) no GitHub. Clique em File > Save As... no seu browser e guarde o ficheiro na pasta do seu projeto
2. Remova o comentário desta linha e altere o caminho para apontar para a pasta do seu projeto

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

(Para descobrir o caminho para a sua pasta num Mac, clique com o botão direito e selecione Obter informações. A janela que aparece mostrará o caminho em 'Localização:')
3. Na Consola Python do QGIS, abra o ficheiro e depois execute o código

```
# Carrega ferramentas que nos permitem controlar mapas e layouts do QGIS a partir deste script

from qgis.PyQt.QtCore import QRectF

from qgis.core import (
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
)

# Carrega math para tratar números, re (regex) para texto, os para navegação de ficheiros

import math, re

import os
```

Aqui é onde a pasta de saída é definida

Usa a biblioteca os para criar uma pasta no seu computador onde guardar as imagens de mapa exportadas.

Será colocada na sua pasta Downloads, dentro de uma subpasta chamada "qgis_images".

Se a pasta já existir, não faz nada – não substituirá nada.

```
out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")
```

```
os.makedirs(out_dir, exist_ok=True)
```

```
out_prefix = out_dir + "/"
```

Para substituir a pasta definida acima, remova o comentário e adapte a linha abaixo

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

```
#set the Coordinate Reference System (CRS) - EPSG:4326 is used for GPS  
#but 27700 is used for UK flood defences so we use this to make the map more rigorous
```

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

```
# Get the currently open QGIS project and store in a variable called project
```

```
project = QgsProject.instance()
```

```
#Using EPSG:27700 means we need to convert lat/long into BNG coordinates  
# first set the source (EPSG:4326) and destination systems (EPSG:27700)
```

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

```
#then create a transformer that uses those two
```

```
transformer = QgsCoordinateTransform(  
    source_crs,  
    dest_crs,  
    project  
)
```

#criar um novo layout de impressão, nesse projeto, guardar numa variável chamada layout

layout = QgsPrintLayout(project)

#preencher com as definições predefinidas

layout.initializeDefaults()

#dar-lhe um nome no QGIS

layout.setName("centres_layout") # Dar um nome ao layout

Adicionar um painel de mapa ao layout – este é o retângulo que vai mostrar o mapa.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Desenhar uma borda em torno do painel de mapa
```

Posicionar o painel de mapa a 20mm da esquerda e 20mm do topo da página.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Definir o painel de mapa com 257mm de largura e 170mm de altura (aproximadamente A4 paisagem sem margens).

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Adicionar o painel de mapa à página do layout
```

Adicionar uma etiqueta de texto ao layout, que mostrará o nome e as coordenadas de cada localização.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Posicionar a etiqueta perto do topo da página (20mm da esquerda, 5mm do topo).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def cria uma função (uma receita) que podemos usar mais tarde. Esta função calcula uma área de mapa retangular centrada num ponto dado. É necessário fornecer quatro ingredientes: x e y do ponto central; e metade da largura/altura (com um valor predefinido para o último item se não for fornecido).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # Se não for fornecida altura, usar o mesmo valor que a largura
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # Quando usada, devolverá um retângulo que pode ser usado para definir o que o mapa mostra.
```

```
    return QRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

Aqui é onde as **localizações** são guardadas

Criar uma lista de localizações a mapear. Cada item tem uma latitude, longitude e um nome.

Começamos com apenas dois itens para testar, mas uma vez que funcione, expandimos para todas as localizações para as quais queremos criar imagens

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Aqui é onde a escala é definida

DEFINIR ESCALA DO MAPA: 1 unidade no mapa equivale a 175000 in reality

MAP_SCALE = 175000

Aqui é onde a saída é definida

Configurar a ferramenta que vai guardar o layout como ficheiro de imagem.

```
exporter = QgsLayoutExporter(layout)
```

```
settings = QgsLayoutExporter.ImageExportSettings()
```

DEFINIR TAMANHO DA IMAGEM DE SAÍDA

1200x795px para uso web, correspondendo à proporção do layout A4 no código anterior

```
settings.imageSize = QSize(1200, 795)
```

```
# Percorrer cada localização na lista e usar a função para criar/exportar uma imagem

for rec in listofdicstocsv:

    # Ler a latitude, longitude e o nome desta localização.

    lat = rec.get('lat', float('nan')) # 'nan' usado como substituto se o valor estiver em falta
    lon = rec.get('long', float('nan'))

    officialname = rec.get('officialname', 'unknown')

    # Se a latitude ou a longitude estiverem em falta, ignorar esta localização e continuar.

    if math.isnan(lat) or math.isnan(lon):

        print(f"skipping '{officialname}': missing lat/long")

        continue

    # Mover a vista do mapa para que fique centrada nesta localização, ampliar usando lon e lat

    transformed_point = transformer.transform( QgsPointXY(lon, lat) )

    # continua...
```

```
# Extrair coordenadas transformadas

x = transformed_point.x()
y = transformed_point.y()

#Criar um retângulo temporário em torno do ponto para o centrar
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Definir uma escala para controlar o zoom. Cada mapa vai usar essa escala.
map_item.setScale(MAP_SCALE)

#adicionar uma etiqueta com essa informação
label.setText( f"{officialname} | scale 1:{MAP_SCALE:,.}" )

#atualizar o mapa antes de exportar
map_item.refresh()

# continua...
```

```
#dar um nome ao ficheiro - remover caracteres problemáticos

safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')

#usar isso para determinar o caminho onde guardar

out_path = ( f"{out_prefix}" f"{safe_name}.png" )

#e exportar para esse caminho

result = exporter.exportToImage( out_path, settings )

#se não for bem-sucedido

if result != QgsLayoutExporter.Success:

    #mostrar uma mensagem de erro

    raise RuntimeError(f"Export failed for {out_path}")

#imprimir uma mensagem no final de cada iteração

print("wrote", out_path)
```

Como faço isso
258 vezes?

Resumo: onde as localizações são guardadas

Criar uma lista de localizações a mapear. Cada item tem uma latitude, longitude e um nome.

Começamos com apenas dois itens para testar, mas uma vez que funcione, expandimos para todas as localizações para as quais queremos criar imagens

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Converter um CSV numa lista de dicionários

1. **Descarregar o CSV das cidades** (clique nos três pontos no canto superior direito)
2. **Escrever esta fórmula na célula J2:**
`=("{'lat': "&B2&", 'long': "&C2&", 'officialname': '"&A2&"'}")"`
3. **Copiar a fórmula para baixo em cada linha. Agora tem uma coluna de dicionários.**
4. **Usar esta fórmula para juntar os resultados e colocar uma vírgula entre cada um**
`=TEXTJOIN(", ", TRUE, J:J)`
5. **No código Python, encontrar os parênteses retos de abertura e fecho [square brackets] que contêm a lista de dois itens, e substituir esses dois itens pela nova lista de dicionários, mantendo os parênteses retos.**

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': -0.529, 'officialname': 'Maldstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': 0.1192, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Batley'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 53.523, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lewes'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornthorpe Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guisely'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrexham'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Elesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7225, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Brintree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Fekstone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbemauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hampstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salsbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Sutton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.4452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tydfil'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middleton'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Ardrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'South Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.112, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Falkirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2763, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Eaerley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŵll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9677, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Chaddle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulson'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Uxminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Usar um slice no ciclo for para testar:

Adicionar [10:20] após o nome da lista significa que só vai percorrer os itens nas posições 10 a 19

```
for rec in listofdicstocsv[10:20]:
```

```
    # Ler a latitude, longitude e o nome desta localização.
```

```
    lat = float(rec.get('lat', float('nan'))) # 'nan' usado como substituto se o valor estiver em falta
```

```
    lon = float(rec.get('long', float('nan')))
```

```
    officialname = rec.get('officialname', 'unknown')
```

```
    # Se a latitude ou a longitude estiverem em falta, ignorar esta localização e continuar.
```

```
    if math.isnan(lat) or math.isnan(lon):
```

```
        print(f"skipping '{officialname}': missing lat/long")
```

```
        continue
```

**Como faço isso
para áreas de
diferentes
tamanhos?**

**Precisamos de escalas
diferentes:**

**Grande conurbação,
área menor**

Vibe coding agora?

Usar meta prompting para pedir à IA para criar o prompt primeiro

Incluir:

- O contexto (jornalismo)
- O objetivo
- **Instruções detalhadas** (nomes específicos, ficheiros, etc.)

É o meu mentor, um jornalista de dados com mais de uma década de experiência na área. Tem **conhecimentos estatísticos avançados** bem como um saudável **ceticismo** ao lidar tanto com dados como com fontes humanas.

Vou pedir-lhe ajuda com algum código - está disponível para me orientar, mas **não quer que eu perca competências nem fique demasiado dependente** de si, por isso os seus conselhos serão sempre concebidos para **obrigar-me a pensar por mim próprio, a aprender novas competências e conceitos, e a praticá-los**.

Quando fornecer código, **adicione comentários a explicar cada passo** como se fosse para uma pessoa sem experiência em programação.

Prefiro **código simples que exija várias linhas em vez de código complexo numa ou duas linhas**.

Assinale quaisquer pressupostos que esteja a fazer sobre os dados ou a minha pergunta. Avise-me se a pergunta não contiver informação suficiente para ser respondida com precisão, ou se o resultado puder ser enganoso sem contexto adicional.

Em anexo está algum Python que gera imagens de mapas centrados em centenas de localizações. **Adapte-o para que agora gere essas imagens em duas escalas diferentes.**

Comente o código para destacar a secção onde isso é feito, para que eu possa ajustá-la e experimentar diferentes níveis de zoom.

Parâmetros

[https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py](https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py)

A secção que define os parâmetros de escala

```
# DEFINIR ESCALAS DO MAPA:
```

```
MAP_SCALES = [  
    175000,    # escala original  
    120000    # versão com mais zoom  
]
```

A secção que usa esses parâmetros

```
# NOVO: PERCORRER MÚLTIPLAS ESCALAS

for MAP_SCALE in MAP_SCALES:

    #Criar um retângulo temporário em torno do ponto para o centrar

    map_item.setExtent(

        rect_around(x, y, 1000, 1000)

    )

    #Aplicar a escala atual

    map_item.setScale(MAP_SCALE)

    #adicionar uma etiqueta com essa informação

    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...e adicionando ao nome do ficheiro

```
#usar isso para determinar o caminho onde guardar
```

```
out_path = (
```

```
    f"{out_prefix}"
```

```
    f"{safe_name}_{MAP_SCALE}.png" # NOVO: ADICIONAR ESCALA AO NOME DO
```

FICHEIRO

```
)
```

**O amigo crítico:
O que a IA deve dizer-
lhe**

Aspetos a considerar

- Converter entre o CRS do mapa base e o shapefile
- Converter entre o tamanho do layout (mm) e imageSize (px)
- Exportar para impressão/resolução de 200dpi
- Categorias de escala em vez de números brutos (e mais de duas)

Experimente isto:
Outros ficheiros
opcionais no repositório

Principais conclusões

- Se tiver de fazê-lo mais do que uma vez...
- Vibe coding é divertido, mas o *mentor* coding é mais gratificante

Recursos adicionais:

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [Vibe Coding Ameaça o Jornalismo: Por Que as Redações Precisam de Governança Agora](#)
- [FT newsletter](#) sobre a metodologia deles que usa IA para ajudar na programação de mapeamento de zonas de risco de inundação